

PLSQL

Exercise 1: Control Structures

Scenario 1:

```
BEGIN

  FOR rec IN
  (
    SELECT CustomerID,
           FLOOR(MONTHS_BETWEEN(SYSDATE,DOB)/12) Age
    FROM Customers
  )
  LOOP
    IF rec.Age > 60 THEN
      UPDATE Loans
      SET InterestRate = InterestRate - 1
      WHERE CustomerID = rec.CustomerID;
      DBMS_OUTPUT.PUT_LINE(
        'Discount Applied to Customer '
        || rec.CustomerID
      );
    END IF;
  END LOOP;

  COMMIT;

END;

/
SELECT LoanID, CustomerID, InterestRate
FROM Loans;
```

Output:

Query result Script output DBMS output Explain Plan SQL history				
  Download ▼ Execution time: 0.004 seconds				
	LOANID	CUSTOMERID	INTERESTRATE	
1	1	1	5	
2	2	3	7	

Scenario 2:

ALTER TABLE Customers

ADD IsVIP VARCHAR2(5);

BEGIN

FOR rec IN

(

SELECT CustomerID,Balance

FROM Customers

)

LOOP

IF rec.Balance > 10000 THEN

UPDATE Customers

SET IsVIP='TRUE'

WHERE CustomerID=rec.CustomerID;

DBMS_OUTPUT.PUT_LINE(

'VIP Customer: '

|| rec.CustomerID

);

ELSE

UPDATE Customers

SET IsVIP='FALSE'

WHERE CustomerID=rec.CustomerID;

```

        END IF;

    END LOOP;

    COMMIT;

END;

/
SELECT CustomerID,Name,Balance,IsVIP
FROM Customers;
Result:

```

Query result Script output DBMS output Explain Plan SQL history					
  Download ▼ Execution time: 0.008 seconds					
	CUSTOMERID	NAME	BALANCE	ISVIP	
1	1	John Doe	1000	FALSE	
2	2	Jane Smith	1500	FALSE	
3	3	Robert King	15000	TRUE	

Scenario 3

```

BEGIN
    FOR rec IN
    (
        SELECT c.Name,
               l.LoanID,
               l.EndDate
        FROM Customers c
        JOIN Loans l
        ON c.CustomerID=l.CustomerID
        WHERE l.EndDate BETWEEN SYSDATE
               AND SYSDATE+30
    )
    LOOP

        DBMS_OUTPUT.PUT_LINE(

```

```
'Reminder: Loan '  
|| rec.LoanID  
|| ' for '  
|| rec.Name  
|| ' due on '  
|| TO_CHAR(rec.EndDate,'DD-MON-YYYY')  
);
```

```
END LOOP;
```

```
END;
```

```
/
```

Output

```
Reminder: Loan 2 for Robert King due on 14-JUL-2026
```

```
PL/SQL procedure successfully completed.
```

Exercise 2: Error Handling

Scenario 1

Create Procedure SafeTransferFunds

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_from_account IN NUMBER,
    p_to_account   IN NUMBER,
    p_amount       IN NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(
            -20001,
            'Insufficient Funds'
        );
    END IF;

    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_account;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_account;
```

```

COMMIT;

DBMS_OUTPUT.PUT_LINE('Transfer Successful');

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK;

        DBMS_OUTPUT.PUT_LINE(

            'Error: ' || SQLERRM

        );

END;

/

SELECT * FROM Accounts;

```

Output:

Query result						Script output	DBMS output	Explain Plan	SQL history
Download						Execution time: 0.016 seconds			
	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED				
1	1	1	Savings	1000	6/24/2026, 10:16:59 AM				
2		2	Checking	1500	6/24/2026, 10:16:59 AM				

Successful Transfer

```

BEGIN
    SafeTransferFunds(1,2,500);
END;

/

```

Output:

```
Transfer Successful
```

```
PL/SQL procedure successfully completed.
```

```
Elapsed: 00:00:00.015
```

Insufficient Funds Test

```
BEGIN
  SafeTransferFunds(1,2,50000);
END;
/
```

Result:

```
Error: ORA-20001: Insufficient Funds
```

```
PL/SQL procedure successfully completed.
```

```
Elapsed: 00:00:00.004
```

Scenario 2

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
  p_empid IN NUMBER,
  p_percent IN NUMBER
)
IS
  v_count NUMBER;
BEGIN
  SELECT COUNT(*)
  INTO v_count
  FROM Employees
  WHERE EmployeeID = p_empid;

  IF v_count = 0 THEN
```

```

        RAISE_APPLICATION_ERROR(
            -20002,
            'Employee ID Not Found'
        );
    END IF;

    UPDATE Employees
    SET Salary = Salary + (Salary * p_percent/100)
    WHERE EmployeeID = p_empid;
    COMMIT;

    DBMS_OUTPUT.PUT_LINE(
        'Salary Updated Successfully'
    );

EXCEPTION

    WHEN OTHERS THEN

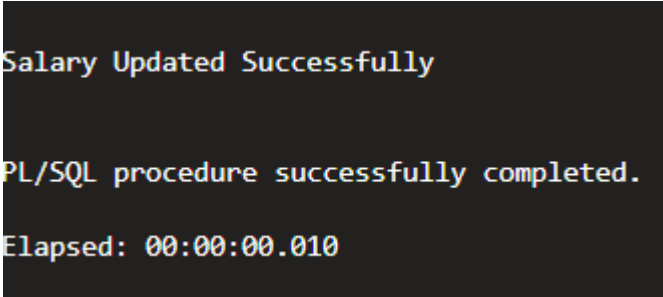
        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );

END;/
Successful Update

BEGIN
    UpdateSalary(1,10);
END;
/

```

Output:



```

Salary Updated Successfully

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010

```

Invalid Employee Test

```
BEGIN
  UpdateSalary(100,10);
END;
/
```

Result:

```
Error: ORA-20002: Employee ID Not Found

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.010
```

Scenario 3

create procedure:

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(

  p_customerid IN NUMBER,

  p_name IN VARCHAR2,

  p_dob IN DATE,

  p_balance IN NUMBER

)

IS

BEGIN

  INSERT INTO Customers

  (

    CustomerID,

    Name,

    DOB,

    Balance,

    LastModified

  )

  VALUES

  (

    p_customerid,
```

```

        p_name,
        p_dob,
        p_balance,
        SYSDATE
    );
COMMIT;
DBMS_OUTPUT.PUT_LINE(
    'Customer Added Successfully'
);
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE(
            'Error: Customer ID Already Exists'
        );
        ROLLBACK;
    WHEN OTHERS THEN

        DBMS_OUTPUT.PUT_LINE(
            'Error: ' || SQLERRM
        );
        ROLLBACK;
END;/

```

Step 2: Successful Insert

```

BEGIN
    AddNewCustomer(
        5,
        'David Miller',
        TO_DATE('1998-06-15','YYYY-MM-DD'),
        3000
    );
END;
/

```

Output

Customer Added Successfully

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.024

Download ▾

Execution time: 0.008 seconds

	CUSTOMERID	NAME	DOB	BALANCE	LASTMODIFIED	ISVIP
1	1	John Doe	5/15/1985, 12:00:00	1000	6/24/2026, 10:16:59 AM	FALSE
2	2	Jane Smith	7/20/1990, 12:00:00	1500	6/24/2026, 10:16:59 AM	FALSE
3	5	David Miller	6/15/1998, 12:00:00	3000	6/24/2026, 10:35:24 AM	(null)
4	3	Robert King	4/10/1950, 12:00:00	15000	6/24/2026, 10:19:36 AM	TRUE

Duplicate ID Test

```
BEGIN
  AddNewCustomer(
    5,
    'Another Customer',
    TO_DATE('2000-01-01','YYYY-MM-DD'),
    5000
  );
END;
/
```

Output

Error: Customer ID Already Exists

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.006

Exercise 3: Stored Procedures

Scenario 1

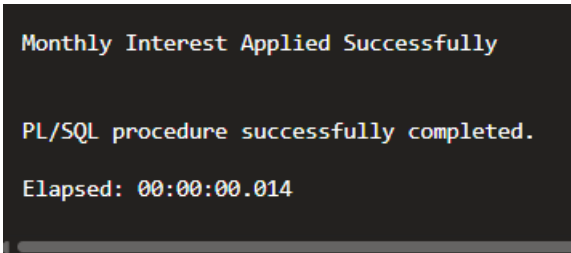
```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01),
        LastModified = SYSDATE
    WHERE AccountType = 'Savings';
    COMMIT;
    DBMS_OUTPUT.PUT_LINE(
        'Monthly Interest Applied Successfully'
    );
END;
```

/

Execute Procedure

```
BEGIN
    ProcessMonthlyInterest;
END;
/
```

Output



```
Monthly Interest Applied Successfully

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.014
```

Query result				Script output	DBMS output	Explain Plan	SQL hist
  Download ▼ Execution time: 0.002 seconds							
	ACCOUNTID	ACCOUNTTYPE	BALANCE				
1	1	Savings	0				
2	2	Checking	2500				

Scenario 2

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(

p_department IN VARCHAR2,

p_bonus_percent IN NUMBER

)

IS

BEGIN

UPDATE Employees

SET Salary = Salary +

(Salary * p_bonus_percent / 100)

WHERE Department = p_department;

COMMIT;

DBMS_OUTPUT.PUT_LINE(

'Bonus Applied Successfully'

);

END;

/

SELECT EmployeeID,

```
Name,  
Department,  
Salary  
FROM Employees;
```

Result:

Query result Script output DBMS output Explain Plan SQL history				
🗑️ ⓘ Download ▾ Execution time: 0.011 seconds				
	EMPLOYEEID	NAME	DEPARTMENT	SALARY
1	1	Alice Johnson	HR	102487
2	2	Bob Brown	IT	60000

Execute Procedure

```
BEGIN  
  UpdateEmployeeBonus('IT',10);  
END;  
/
```

Output

```
Bonus Applied Successfully  
  
PL/SQL procedure successfully completed.  
  
Elapsed: 00:00:00.013
```

Verify

```
SELECT EmployeeID,  
       Name,  
       Department,  
       Salary
```

FROM Employees;
output:

Query result Script output DBMS output Explain Plan SQL history					
  Download ▾ Execution time: 0.008 seconds					
	EMPLOYEEID	NAME	DEPARTMENT	SALARY	
1	1	Alice Johnson	HR	102487	
2	2	Bob Brown	IT	66000	

Scenario 3

```
CREATE OR REPLACE PROCEDURE TransferFunds(
```

```
    p_from_account IN NUMBER,
```

```
    p_to_account IN NUMBER,
```

```
    p_amount IN NUMBER
```

```
)
```

```
IS
```

```
    v_balance NUMBER;
```

```
BEGIN
```

```
    SELECT Balance
```

```
    INTO v_balance
```

```
    FROM Accounts
```

```
    WHERE AccountID = p_from_account;
```

```
    IF v_balance < p_amount THEN
```

```
        DBMS_OUTPUT.PUT_LINE(
```

```
            'Insufficient Balance'
```

```
        );
```

ELSE

UPDATE Accounts

SET Balance = Balance - p_amount

WHERE AccountID = p_from_account;

UPDATE Accounts

SET Balance = Balance + p_amount

WHERE AccountID = p_to_account;

COMMIT;

DBMS_OUTPUT.PUT_LINE(

'Transfer Successful'

);

END IF;

END;

/

Step 2: Check Balances

SELECT AccountID,

Balance

FROM Accounts;

Before

Query result		Script output	DBMS output	Explain Plan	SQL histor
		Download	Execution time: 0.004 seconds		
	ACCOUNTID	BALANCE			
1	1	0			
2	2	2500			

Step 3: Execute Procedure

```
BEGIN
    TransferFunds(1,2,500);
END;
/
SELECT AccountID,
       Balance
FROM Accounts;
```

Output

Query result				Script output	DBMS output	Explain Plan	SQL history
  Download ▾				Execution time: 0.001 seconds			
	ACCOUNTID		BALANCE				
1	1		500				
2	2		3000				

Test Insufficient Balance

```
BEGIN
    TransferFunds(1,2,10000);
END;
/
```

Output

```
Insufficient Balance

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.004
```